PLAN

- 1. Introduction programmation mobile (Android)
- 2. Environnement de Développement
- 3. Cycle de Vie d'une Activité
- 4. Interfaces graphiques
- 5. Les Intents
- 6. Persistance des données
- 7. Capteurs sous Android
- 8. Programmation Concurrente

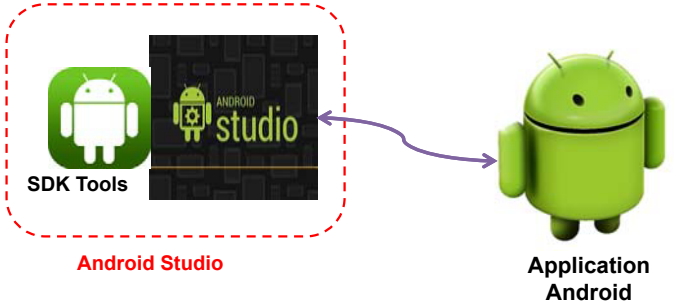
Environnement de Développement

- On peut développer sur Android sous Linux, Windows et MacOS avec :
  - ~~Eclipse + un plugin~~;
  - Android-Studio.
- Ces deux solutions utilisent le SDK d'Android qui contient :
  - Les librairies Java d'Android ;
  - Des outils de développement ;
  - un émulateur pour tester vos applications ;
  - des images du système Android...
- Il n'est pas nécessaire de télécharger le SDK séparément si on va utiliser Android Studio.

Environnement de Développement 1/2 (Obsolete)



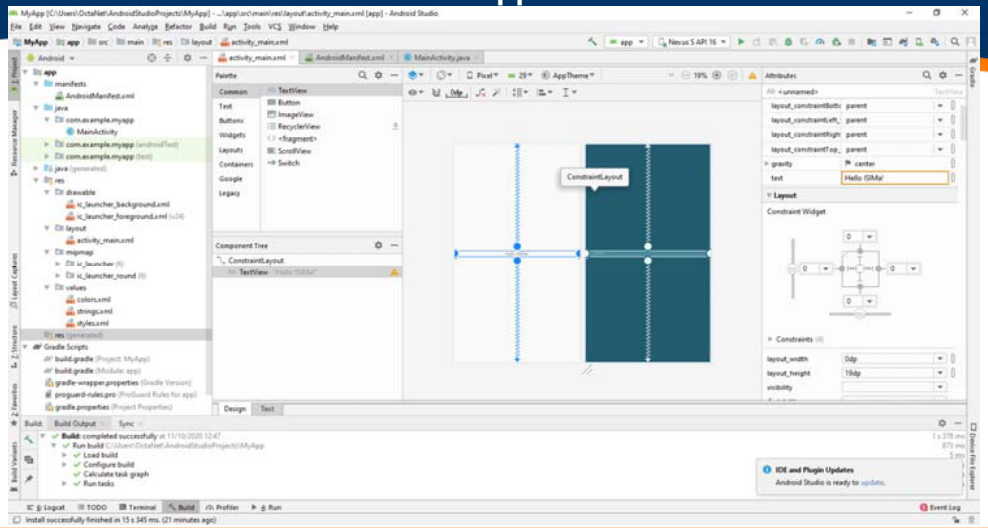
Environnement de Développement 2/2



Environnement de Développement : Android Studio

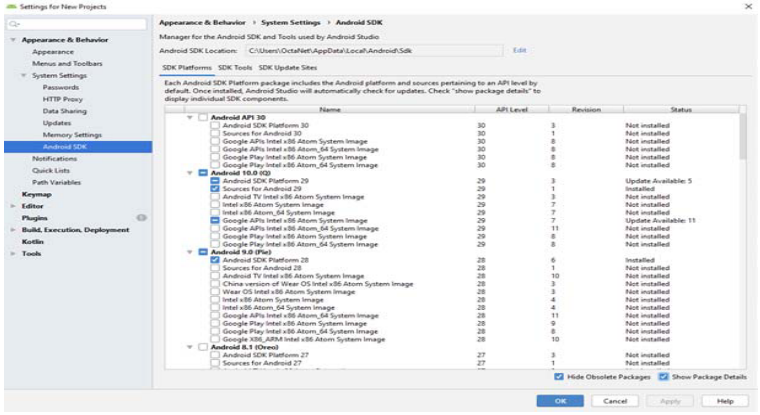
- Depuis mi-2015, il est recommandé d'utiliser Android Studio (l'intégration de fonctionnalités de développement Android).
- Avantages:
  - Meilleure intégration du SDK dans Android Studio
  - Puissance de l'IDE IntelliJ
  - Meilleure gestion des dépendances avec *gradle*
- Pour installer Android Studio :
  - Télécharger le logiciel via le site d'Android ;
  - Extraire l'archive dans un répertoire ;
  - (Linux) Exécuter le fichier studio.sh du répertoire bin / (windows) Lancer le fichier exécutable exe ;
- À son premier démarrage, Android Studio :
  - Télécharge le sdk ;
  - Va vous proposer de configurer une machine virtuelle.

Environnement de Développement : Android Studio



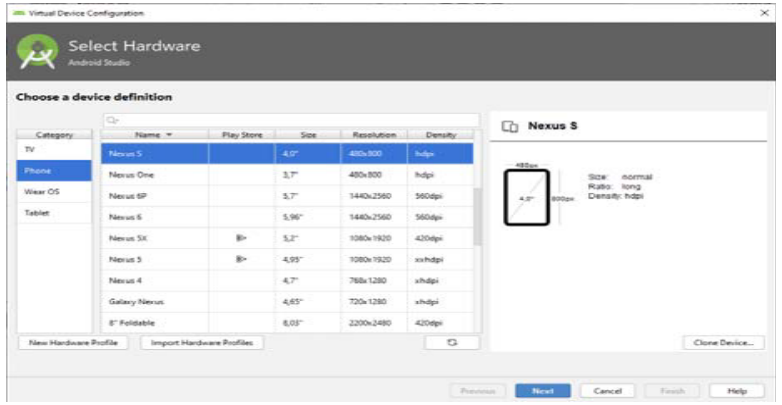
Environnement de Développement : Android SDK

- Vous pouvez exécuter *Android SDK Manager*, en lançant l'exécutable *android* qui se trouve dans le répertoire *tools* du SDK :



Environnement de Développement : AVD Manager

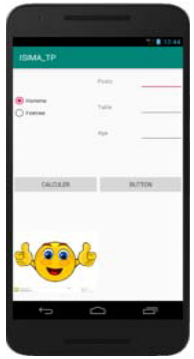
- Pour tester un programme sur votre ordinateur, il sera également nécessaire de créer une machine virtuelle à l'aide de l'*AVD Manager* :



Environnement de Développement : Emulateur

- L'émulateur vous permet de tester votre application :





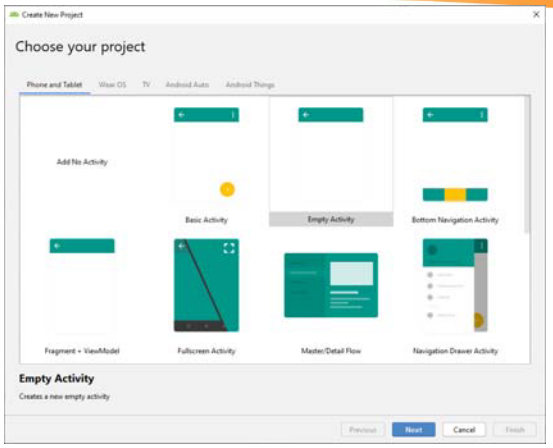
- Les ROMs Intel permettent de bénéficier de l'accélération matérielle.

Environnement de Développement : SmartPhone

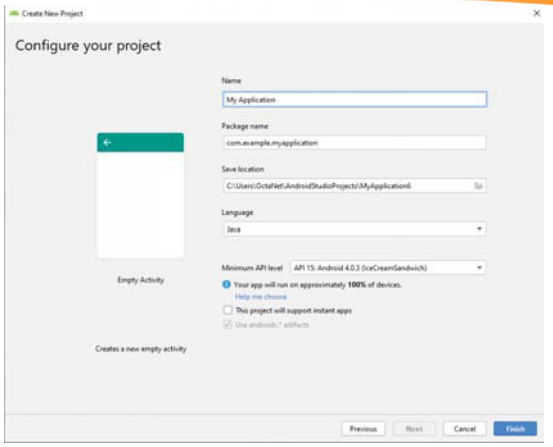
- Votre SmartPhone vous permet de tester votre application (en activant l'options de développement) :



Environnement de Développement : Nouveau Projet



Environnement de Développement : Nouveau Projet



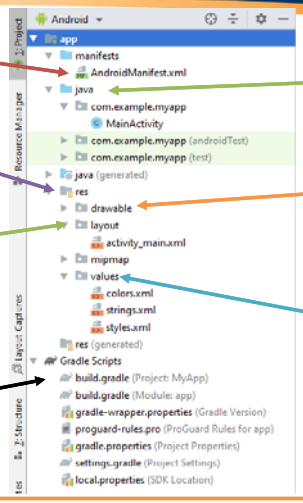
Environnement de Développement : Architecture d'un projet

Fichier de contrôle de l'application, décrivant sa nature, ses composants et les permissions associées

Ressources utilisées par l'application

Fichiers XML représentant l'interface d'une page ou d'une partie de page de votre application

Des fichiers de configuration:
- build.gradle (2 instances): règles de dépendance et de compilation
- settings.gradle: liste de toutes les applications à compiler (si plusieurs)



Code source Java des applications

Images (PNG, JPEG ou GIF) et fichiers XML décrivant les formes

Dossier contenant les fichiers décrivant des valeurs utilisées dans l'application. Ex : strings.xml pour les chaînes, arrays.xml pour les tableaux…

Environnement de Développement : AndroidManifest.xml

- **AndroidManifest.xml est le fichier de configuration XML de l'application.**
  - Il ne doit pas être renommer, ni déplacer
- **C'est un fichier indispensable à chaque application qui décrit entre autres :**
  - le point d'entrée de votre application (quel code doit être exécuté au démarrage de l'application) ;
  - Le nom de l'application, son icone, son package (l'identifiant unique sur le Play Store)
  - Définit les composants qui constituent ce programme ;
  - les permissions nécessaires à l'exécution du programme :
    - Les permissions matérielles (accès à l'appareil photo, GPS, ...).
    - Les permissions d'interactions entre les applications (accès à Internet, répertoire téléphoniques, galerie des photos…)

Environnement de Développement : AndroidManifest.xml

- **<application> : un seul et unique noeud application**
- **<uses-permission> : l'utilisateur se verra demander l'autorisation d'utiliser l'ensemble des fonctions liées à ces permissions**
- **<activity> : Déclare une activité présentée à l'utilisateur**
- **<service> : Déclare un composant de l'application en tant que service.**
- **<receiver> : Déclare un récepteur d'objets Intent. Cet élément permet à l'application de recevoir ces objets alors qu'ils sont diffusés par d'autres applications ou par le système.**
- **<provider> : Déclare un fournisseur de contenu qui permettra d'accéder aux données gérées par l'application.**

Environnement de Développement : AndroidManifest.xml

- **Le fichier AndroidManifest.xml déclare l'ensemble des éléments de l'application.**

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android=http://schemas.android.com/apk/res/android package="com.example.myapp">

<uses-permission android:name="android.permission.INTERNET"></uses-permission>
<uses-sdk android:minSdkVersion="15"></uses-sdk>

<application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/AppTheme">
        <activity android:name=".SecondActivity"    android:exported="false" />

        <activity android:name=".MainActivity"    android:exported="true">
                <intent-filter>
                        <action android:name="android.intent.action.MAIN" />
                         <category android:name="android.intent.category.LAUNCHER" />
                </intent-filter>
        </activity>
    </application>
</manifest>
```

Environnement de Développement : AndroidManifest.xml

Détails du manifest

```
<?xml version="1.0" encoding="utf-8"?>
<manifest>
    <uses-sdk />
    <uses-permission />
    <application>
        <activity>
            <intent-filter>
                <action />
                <category />
                <data />
            </intent-filter>
        </activity>
        <service>
            <intent-filter>
            . . .
            </intent-filter>
        </service>
        <receiver>
            <intent-filter>
            . . .
            </intent-filter>
        </receiver>
        <provider>
            <grant-uri-
             permission />
        </provider>
        <uses-library />
    </application>
</manifest>
```

Général

Pour chaque service

Pour chaque écouteur d'intentions diffusées

Pour chaque fournisseur de contenu

Pour chaque activité

Environnement de Développement : MainActivity.java

```java
import androidx.appcompat.app.AppCompatActivity;

import android.os.Bundle;
import android.widget.Toast;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        setContentView(R.layout.activity_main);

        Toast.makeText(this, "application crée", Toast.LENGTH_SHORT).show();
    }

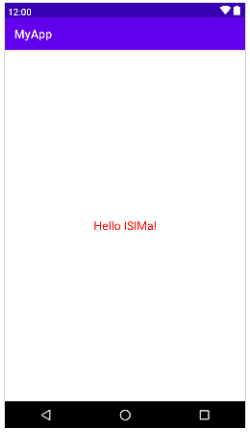
}
```


Environnement de Développement : layout.xml

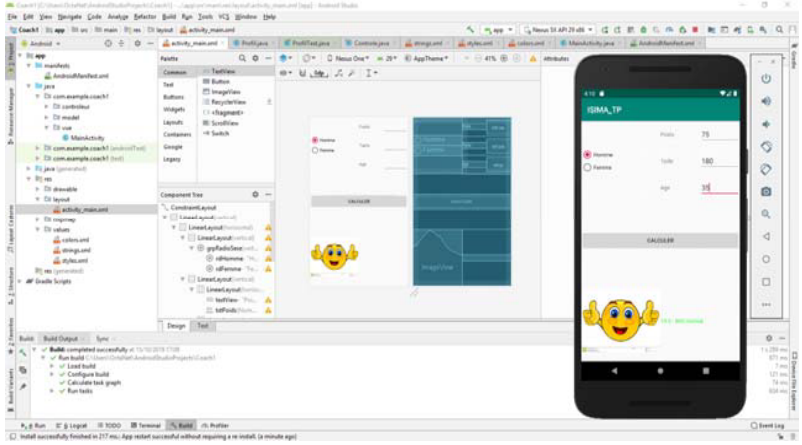
```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    tools:context=".MainActivity">

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:textSize="20sp"
        android:textColor="#ff0000"
        android:text="Hello ISIMa!"/>

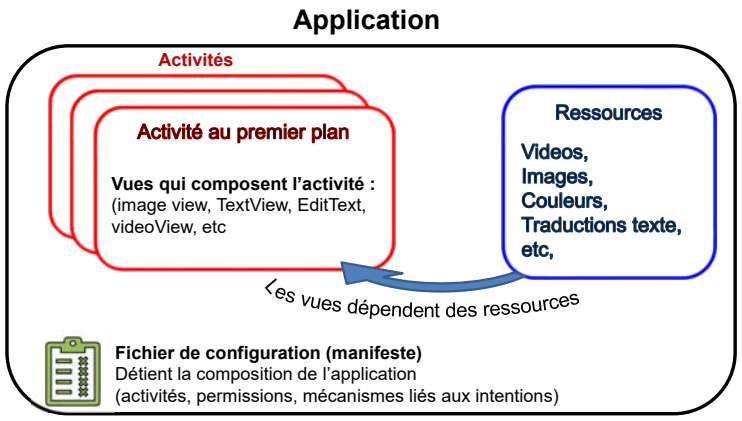
</LinearLayout>
```



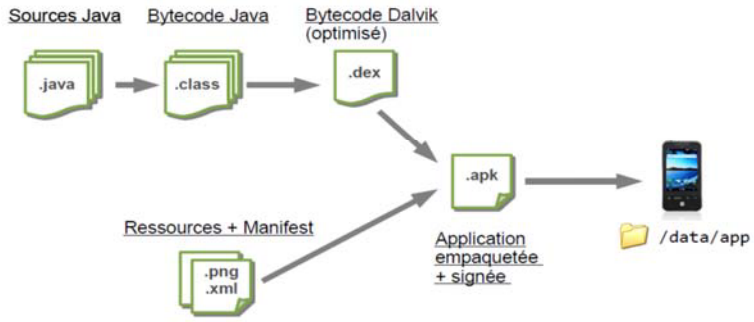
Environnement de Développement : Exemple d'application



Architecture d'une application Android



Développement d'une application



PLAN

- 1. Introduction programmation mobile (Android)
- 2. Environnement de Développement
- **3. Cycle de Vie d'une Activité**
- 4. Interfaces graphiques
- 5. Les Intents
- 6. Persistance des données
- 7. Capteurs sous Android
- 8. Programmation Concurrente

Présentation d'une Activité

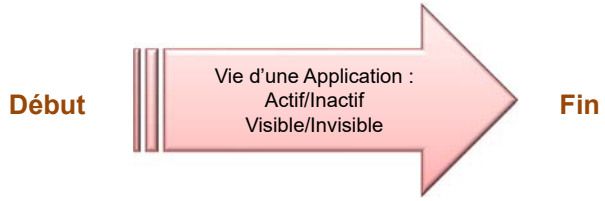
- **Une activité est un composant d'une application qui fournit un écran avec lequel l'utilisateur peut interagir.**
- **Une application contient plusieurs activités**
- **Une activité = Contexte + une Interface graphique**

- **Question : que se passe-t-il si on reçoit un SMS alors que l'utilisateur manipule l'application mobile?**
  - **L'activité de l'application mobile passe d'un état à un autre.**

C'est le Cycle de Vie d'une Activité

Cycle de Vie d'une Activité

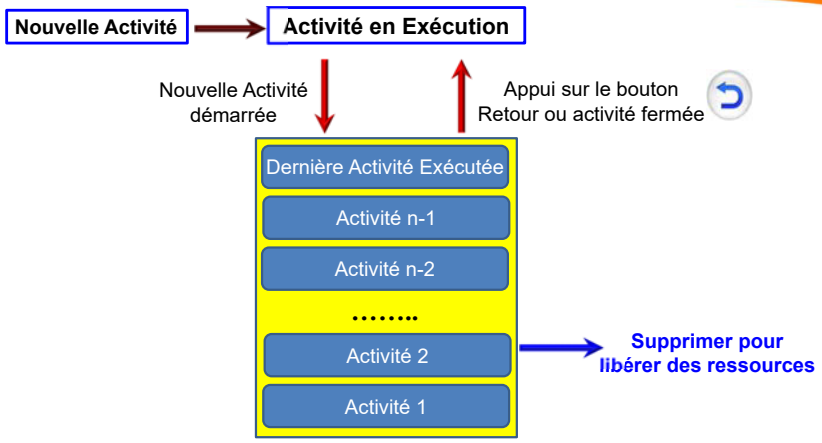
- Les activités d'une application ont un cycle de vie
  - Un début quand Android les instancie pour répondre aux Intents
  - Une fin quand les instances sont détruites
  - Entre les deux, où ils peuvent être:
    - Visibles ou invisibles
    - Actifs ou inactifs



Cycle de Vie d'une Activité

- Les activités dans une application sont gérées sous forme de Pile
  - Quand une nouvelle activité démarre, elle est placée en haut de la pile et devient l'activité en exécution
  - L' activité précédente reste en dessous dans la pile,
  - L' activité précédente ne revient au premier plan que si la nouvelle activité est fermée
  - Si l'utilisateur clique sur le bouton *Retour* du téléphone l'activité suivante dans la pile (*l' activité précédente*) devient active

Cycle de Vie d'une Activité



Cycle de Vie d'une Activité

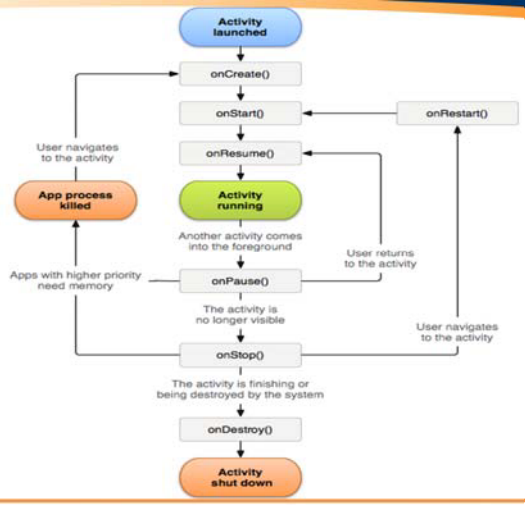
- Active ou en Exécution
  - Elle est au premier plan de l'écran (donc en haut de la pile)
  - C'est l'activité ciblée par les actions de l'utilisateur
- En Pause
  - A perdu le focus, mais est encore partiellement visible
  - Une autre activité est en haut de la pile, mais elle est soit transparente, soit ne couvre pas tout l'écran
- Arrêtée
  - Complètement recouverte par une autre activité
  - Ses informations sont encore chargées, mais elle n'est plus visible
  - Peut être détruite par le système s'il est besoin de mémoire

Cycle de Vie d'une Activité

- Quand une activité passe d'un état à un autre, le framework Android appelle les méthodes de transition correspondantes :
  - void onCreate (Bundle savedInstanceState) : obligatoire
  - void onStart ()
  - void onResume ()
  - void onRestart ()
  - void onPause () : recommandée
  - void onStop ()
  - void onDestroy ()

- Quand l'utilisateur reçoit un appel téléphonique, l'activité courante passe dans l'état OnPause ensuite, Stopped, lorsque l'appel téléphonique est terminé, cette activité revient dans Started puis Resumed.

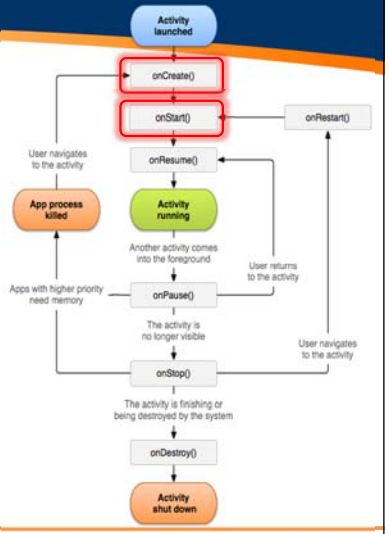
Cycle de Vie d'une Activité



Cycle de Vie d'une Activité

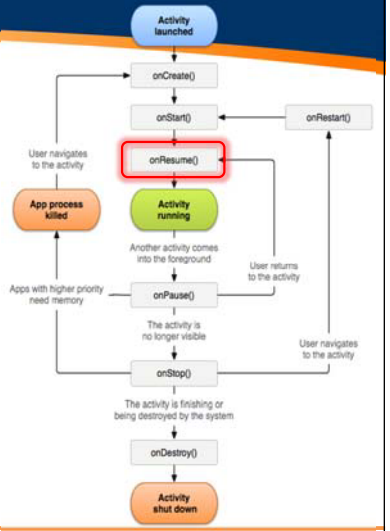
- onCreate
  - Appelée à la création d'une activité
  - Initialisation de tous les éléments
  - Un bundle est passé à cette méthode, contenant l'état précédent de l'activité
  - Toujours suivie de onStart
  - L'instruction setContentView (R.layout.activity_main); permet d'affecter l'interface graphique à l'activité qui est décrite dans le fichier activity_main.xml.

- onStart
  - Appelée juste avant que l'activité ne devienne visible
  - Suivie de
    - onResume si l'activité revient en premier plan



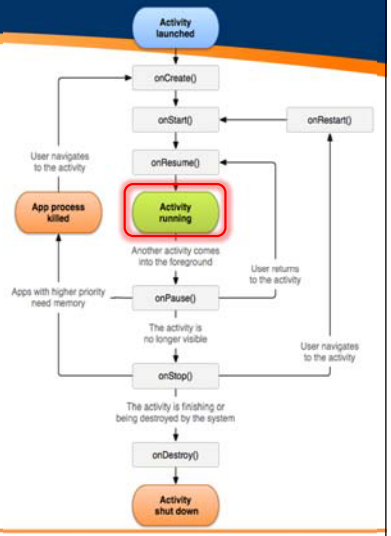
Cycle de Vie d'une Activité

- onResume
  - Appelée juste avant que l'activité ne commence a interagir avec l'utilisateur
  - A ce point, l'activité est en haut de la pile
  - Toujours suivie de onPause



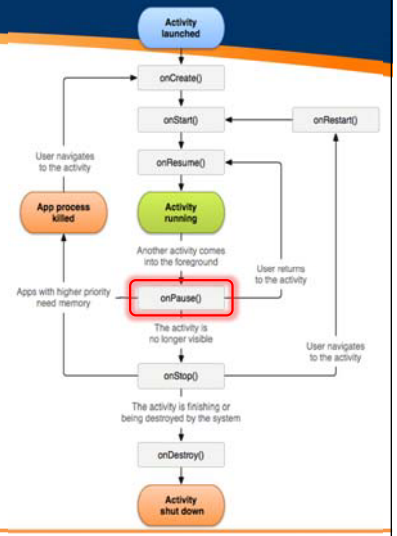
Cycle de Vie d'une Activité

- **Activity running**
  - L'application est en exécution
  - L'utilisateur est actuellement en interaction avec l'application
  - Lorsqu'un évènement externe arrive l'activité bascule vers l'état **onPause**



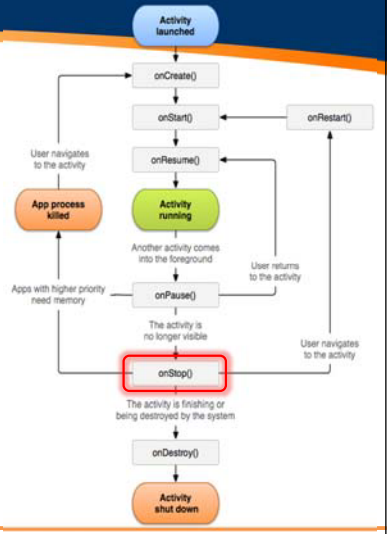
Cycle de Vie d'une Activité

- **onPause**
  - Appelée quand le système va démarrer une autre activité
  - Utilisée typiquement pour :
    - Enregistrer les données non sauvegardées
    - Arrêtées les animations ou tout ce qui consomme de la mémoire...
  - Ne doit pas consommer beaucoup de temps, car l'activité suivante ne va démarrer que si cette méthode prend fin,
  - Suivie de :
    - **onResume** si l'activité est rechargée en premier plan
    - **onStop** si l'activité devient invisible. Dans cet état, l'activité peut être détruite par le système



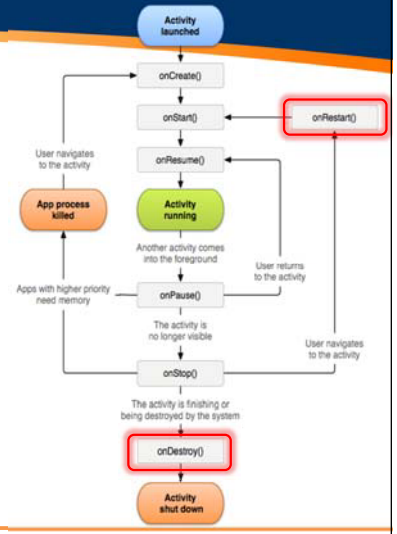
Cycle de Vie d'une Activité

- **onStop**
  - Appelée quand l'activité n'est plus visible à l'utilisateur
  - Peut arriver si:
    - L'activité est détruite
    - Une autre activité a repris son exécution et l'a recouverte
  - Suivie par:
    - **onRestart** si l'activité recommence à interagir avec l'utilisateur
    - **onDestroy** si l'activité va disparaître
  - Dans cet état, l'activité peut être détruite par le système



Cycle de Vie d'une Activité

- **onRestart**
  - Appelée quand l'activité va redémarrer après avoir été stoppée
  - Toujours suivie de **onStart**
- **onDestroy**
  - Appelée quand l'activité est détruite
  - Dernier appel que l'activité va recevoir
  - Peut intervenir si:
    - L' activité se termine (appel de finish)
    - Le système détruit temporairement cette instance de l'activité pour gagner de l'espace
  - On peut distinguer entre ces deux scénarios avec la méthode **isFinishing()**
  - Dans cet état, l'activité peut être détruite par le système



Cycle de Vie d'une Activité : Sauvegarde et Restauration du Contexte

- Il faudra personnaliser ces deux méthodes pour sauvegarder et restaurer les données.

```
@Override
protected void onSaveInstanceState(Bundle outState) {
    super.onSaveInstanceState(outState);
}

@Override
protected void onRestoreInstanceState(@NonNull Bundle savedInstanceState) {
    super.onRestoreInstanceState(savedInstanceState);
}
```